第九届

全国大学生集成电路创新创业大赛

报告类型：仿真报告

参赛杯赛：北方华创杯

作品名称：基于混合启发式蚁群算法的半导体晶圆调度策略

队伍编号：CICC0901149

团队名称：同芯合 E

目录

1 引言 4
1.1 半导体制造调度问题的行业背景和经济效益 4
1.2 现有方法的局限性分析 4
1.3 赛题技术创新点 4
2 问题分析 5
3 符号说明 6
4 数学模型 7
4.1 约束条件 7
4.1.1 资源互斥约束 7
4.1.2 模块锁互斥 8
4.1.3 PM 模块内禁止超片 8
4.1.4 LLA 和 LLB 模块槽位约束 8
4.1.5 操作合法性约束： 9
4.1.6 Just-In-Time(JIT)限制 9
4.1.7 清洁机制触发条件 9
4.1.8 时间消耗模型 10
4.2 目标函数 10
4.2.1 最小化整体生产时间 10
4.2.2 最大化晶圆吞吐量 10
4.2.3 最小化 JIT 违规次数 11
4.2.4 多目标函数 11
5 算法设计 11
5.1 多线程并发机制 11
5.1.1 锁的设置 11
5.1.2 仿真流程概述： 12
5.2 智能调度策略-混合启发式蚁群算法 12
5.2.1 状态转移概率（调度选择规则） 12

5.2.2 信息素更新公式 13
5.2.3 动态调参机制 14
5.3 混合调度策略 15
5.4 死锁检测与预防 16
5.4.1 构建资源分配图 16
5.4.2 基于牺牲函数的晶圆选择 16
5.4.3 晶圆回滚机制 17
6 求解结果 18
6.1 任务 1 18
6.1.1 任务 a 18
6.1.2 任务 b 18
6.1.3 任务 c 19
6.1.4 任务 d 19
6.2 任务 2 20
6.2.1 任务 a 20
6.2.2 任务 b 20
6.2.3 任务 c 21
6.2.4 任务 d 21
6.3 参数自适应结果分析(以任务 2 的 d 为例) 22
6.3.1 实时调度监控图曲线说明 22
6.3.2 ACO 参数自适应过程图曲线说明 23
6.4 多目标优化过程中目标函数的演化趋势 24
7 算法性能分析 25
7.1 主控调度流程 25

7.2 机械臂调度 26

7.3 ACO 蚁群算法调度 26

7.4 死锁检测机制 26

7.5 Just-in-time 检查 26

7.6 整体复杂度 26

8 算法优化前后对比 26

9 模型改进 28

10 引用文献 28

1 引言

1.1 半导体制造调度问题的行业背景和经济效益

半导体制造是典型的高投入、高技术密集型产业，其中晶圆调度作为制造厂的核心环节，直接影响芯片加工厂的效益。研究表明，优化调度算法可将生产周期缩短至 15%-30%，对于实现“准时生产”（Just-In-Time,JIT）[3]和发展高产能半导体设备具有重要意义。面对全球芯片供应紧张的情况下，优化晶圆加工厂的调度算法既能创造显著的经济效益，又能对整个产业链的稳定做出巨大贡献。

1.2 现有方法的局限性分析

当前半导体调度主要面临的三大挑战：

(1) 组合爆炸问题：在存在 n 片晶圆和 m 个处理模块的情形下，调度问题的空间复杂度可达 $O(n!m^n)$；

(2) 动态约束：现有研究多假设设备状态变化服从确定性规律 [4]，而实际生产中设备退化具有不确定性，包括 JIT 时间限制、设备状态切换延迟等，使得调度求解更具实时性和复杂性；

(3) 多目标冲突：调度既要考虑使生产周期（Makespan）、产线吞吐量（Throughput）、能耗（Energy Consumption）等相互制约多目标同时达到最优。

在现有算法中，数学规划、Petri 网和分支定界法等忽略了真空锁状态转换对于内外机械手搬运晶圆的影响 [6]；基于规则的调度（Rule-based）方法则会在任务较为复杂时受到非线性约束的影响 [2]；元启发算法在解决晶圆制造调度问题时，其应用会随着问题规模的扩大而受到制约 [1]。综合其他研究表明，传统调度方法因动态约束难以满足 JIT 要求，且死锁事件频发，尚无法满足高效、智能化调度的实际需求。

1.3 赛题技术创新点

本文提出三项创新：

(1) 混合启发式 ACO 架构，融合线程池并发执行与动态参数自适应机制；

(2) 自适应目标权重策略，通过系统负载率、JIT 违规率等指标实时调整优化方向；

(3) 基于牺牲代价模型的死锁回滚算法。

实验表明，本方法在尽可能满足 JIT 合规的情况下，将晶圆调度时间大大缩短，为半导体智能调度提供了一套可参考的解决方案。

2 问题分析

半导体设备作为芯片制造厂的重要部件，晶圆在各类加工设备之间流转的效率，直接影响整个生产线的生产周期以及加工厂的效益。为了精确匹配生产与客户需求，实现零库存和消除浪费，此加工过程引入了 Just-In-Time 机制，进而提高了生产效率和降低了成本。

该任务需根据所给的任务模式和工艺路径完成不同任务的晶圆调度，需避免模块冲突和死锁，保证 PM 模块内部的加工顺序，处理特定情况下的清洁需求，并满足 JIT 控制等约束条件，最终实现晶圆调度时间的最小化。因此我们团队建立了数学规划模型，并采用混合启发式蚁群算法来求解最短调度时间。

本项目的调度系统涉及多个加工模块，其功能描述如表一所示。

表一 系统的模块作用说明

模块名称	功能描述
LP_i，$\mathrm{i}=1,2,3$	3 个晶圆盒，晶圆从 LP 取出或放回
AL	晶圆对准模块
LLA, LLB	真空/大气转换模块
LLC, LLD	晶圆放置区
TM_i，$\mathrm{i}=1,2,3$	机械手抓取晶圆
PM_i，$\mathrm{i}=1,2,...,10$	晶圆加工模块

这些模块通过机械手进行连接，形成复杂的晶圆传输网络，且每个模块都有转换时间。此加工过程容易产生资源竞争与死锁问题，为此我们引入了信号量并构建图通过深度优先搜索算法检测是否有环路进而判断是否会出现死锁情况。

潜在死锁场景：

(1) 多个晶圆等待 S1 空槽，但 S2 被占用无法转移；

(2) 双臂机械手臂指向固定，两端手臂可能产生互相阻塞；

(3) PM 模块清洁等待晶圆取出，晶圆等待清洁完成；

(4) 第一个晶圆加工完成需要等待取出，而机械臂等待把另一个晶圆放入。

图 1 展示了晶圆加工的基本流程，清晰呈现了晶圆的加工过程。

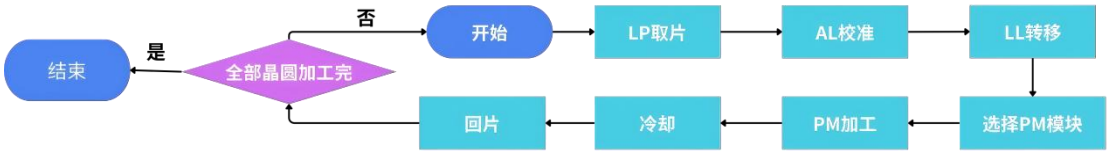

图 1 系统框架图

3 符号说明

符号	说明
W_{ij}， i=1, 2, 3， j=1, 2, . . . , 25	晶圆集合
$M=\{LP,AL,LL,PM,TM\}$	模块集合
$P(w)$	晶圆 w 的加工路径（工艺流程）
$x_{w,m}(t)$	时刻 t,晶圆 w 是否占用模块 m，取值 0，1
$lock_m(t)$	时刻 t，模块 m 是否被锁定，取值 0，1
$s_{w,m}$	晶圆 w 开始在模块 m 加工时间
$e_{w,m}$	晶圆 w 完成在模块 m 加工时间
t_{open},t_{close}	开门/关门所需时间（固定 1s）
t_{pick},t_{place}	机械臂取片/放片时间（固定 4s）
t_{pump},t_{vent}	LLA 和 LLB 泵气/通气时间（15s/20s）

$t_{process}^{m}, t_{stay}^{m}$	PM 模块 m 的加工时间，晶圆加工完在 PM 的驻留时间
Δt_{w}	晶圆 w 在两次操作之间的驻留时间
$JIT_threshold$	JIT 限制即系统允许的最大转移时间
a_{w_i,PM_j}	晶圆 w 到达 PM 的时间
$S_i^{LL}(t) \in \{None, wafer_id\}$，$\mathrm{LL} \in \{LLA, LLB\}, i = 1,2$	时间 t 时刻槽位 S 的晶圆占用状态
$state^{LL}(t) \in \{atm, vacuum, switching_to_atm, switching_to_vacuum\}, \mathrm{LL} \in \{LLA, LLB\}$	时间 t 的 LL 的物理状态
$op^{LL}(t)$	时间 t 的操作动作
$e_m^{last_end}$	PM 模块的上次加工结束时间
$P_{prev}(m), P_{cur}(m)$	模块上次加工路径与当前加工路径
$N_{processed}(m)$	当前连续加工的晶圆数量

4 数学模型

本问题的目标是建立晶圆调度的数学模型，需要满足时间约束、资源互斥规则、晶圆按照指定路径加工，同时满足 JIT 控制等要求实现最优的晶圆调度。

4.1 约束条件

4.1.1 资源互斥约束

每个模块只能被一个晶圆占用：

$$\sum_{w \in W} x_{w,m}(t) \le 1, \forall m \in M, \forall t$$

代表同一时间内一个模块最多只允许一片晶圆使用

4.1.2 模块锁互斥

模块锁的状态应与晶圆占用状态一致：

$$lock_m(t) = \max_{w \in W} x_{w,m}(t)$$

即如果模块被某片晶圆占用，则此模块处于锁定状态

4.1.3 PM 模块内禁止超片

同一 PM 下，相同路径、同一工艺步骤的晶圆，晶圆从小到大进入 PM，因此 PM 模块内部加工顺序必须满足先进先出原则：

若：

$$a_{w_1,PM} < a_{w_2,PM}$$

则：

$$s_{w_1,PM} < s_{w_2,PM}$$

即先进入设备的晶圆先进入 PM。

4.1.4 LLA 和 LLB 模块槽位约束

大气->真空

当：

$$state^{LL}(t) = atm$$

且准备向 TM2 方向转移晶圆。

则：

$$state^{LL}(t+\delta) = switching_to_vacuum$$

持续时间 15s，转换后：

$$state^{LL}(t+15) = vacuum$$

真空->大气

当：

$$state^{LL}(t) = vacuum$$

且准备从 TM1 方向送入晶圆。

则：

$$state^{LL}(t+\delta)=switching_to_atm$$

持续 20s，转换后：

$$state^{LL}(t+20)=atm$$

4.1.5 操作合法性约束：

状态切换期间 TM 不可以操作：

$state^{LL}(t)\in\{switching_to_atm, switching_to_vacuum\}\Rightarrow$ 放入/取出动作暂停

来自 TM2（真空）侧将晶圆放入 LL（向 S2）条件：

$$(state^{LL}(t)=vacuum)\Lambda(S2^{LL}(t)=None)$$

来自 TM1（大气）侧将晶圆放入 LL（向 S1）条件：

$$(state^{LL}(t)=atm)\Lambda(S1^{LL}(t)=None)$$

来自 TM2（真空）侧将晶圆从 LL（从 S2）中取出条件：

$$(state^{LL}(t)=vacuum)\Lambda(S2^{LL}\neq None)$$

来自 TM1（大气）将晶圆从 LL（从 S1）中取出条件：

$$(S1^{LL}(t)\neq None)\Lambda\text{不在切换状态}$$

单一 LL 模块只能执行一个操作：

$$\forall t,\text{不允许同时有}\mathrm{op}^{LL}(t)\in\{pump, vent, load, unload\}$$

4.1.6 Just-In-Time(JIT)限制

在 PM 驻留时间不得超过 15s：

$$t_{stay}^{m}\leq 15$$

每片晶圆在模块之间驻留时间不能超过 30s:

$$\Delta t_w=s_{w,m_{next}}-e_{w.m_{prev}}\leq JIT_threshold$$

否则判定 JIT 违规。

4.1.7 清洁机制触发条件

以任务 1 为例：

a：PM 空闲 80s：30s

b:工艺路径切换：200s

c:已加工 13 片晶圆：300s

判断时间 t 时晶圆是否需要清洁：

$$clean_flag_m(t)=\begin{cases}1, if(t-e_m^{last_end}>80)or(P_{prev}(m)\neq P_{cur}(m))or(N_{processed}(m)\geq 13)\\0, other\end{cases}$$

清洁期间，PM 模块禁止其他晶圆占用：

$$\sum_{w\in W}x_{w,m}(t)=0, clean_flag_m(t)=1$$

即清洁过程中，不允许任何晶圆使用清洁中的 PM 加工。

4.1.8 时间消耗模型

在每次进入晶圆模块进行加工时，总占用时间包括开门、加工、关门：

$$OccupyTime_{w,m}=t_{open}+t_{process}^{m}+t_{close}$$

若晶圆加工完从模块取出后又有新的晶圆进入则会省去一组开关门时间。

机械臂 TM 运转晶圆时，总占用时间包括取片、移动、放片：

$$TransferTime_{w,m}=t_{pick}+\text{路径转移耗时}+t_{place}$$

4.2 目标函数

4.2.1 最小化整体生产时间

定义整体生产完成时间 T_{total} 为最后一片晶圆加工完成并回到晶圆盒的时间，设 e_w^{final} 为最后一片晶圆 w 放回 LP 的时间。则此目标函数是：

$$\min T_{\text{total}}=\min(\max_{w\in W}e_w^{final})$$

即最后一片晶圆完成的时间越早越好。

4.2.2 最大化晶圆吞吐量

晶圆吞吐量（Throughput），即单位时间内完成加工的晶圆数量：

$$Throughput=\frac{|W|}{T_{total}}$$

$|W|$:晶圆总数

目标函数可表示为：

$$\max Throughput$$

即在同一段时间内，能加工更多的晶圆，提高产能效率。

4.2.3 最小化 JIT 违规次数

$$JIT_w = \begin{cases} 1, \text{超时违规} \\ 0, \text{未违规} \end{cases}$$

目标函数可表示为：

$$\min(Total_JIT_violations = \sum_{w \in W} JIT_w)$$

即尽量让所有晶圆严格控制驻留时间，避免超时。

4.2.4 多目标函数

在实际调度中，需要在多个目标间进行综合平衡，因此我们构建了加权综合优化目标：

$$\left.\begin{aligned} &\min T_{\text{total}} = \min(\max_{w \in W} e_w^{final}) \\ &Throughput = \frac{|W|}{T_{total}} \\ &\min(Total_JIT_violations = \sum_{w \in W} JIT_w) \end{aligned}\right\} \min(\alpha_1 \times T_{total} - \alpha_2 \times Throughput + \alpha_3 \times Total_JIT_violations)$$

不同场景下需要根据实际需求调整权重，但最小化整体生产时间是占主要部分。我们设置初始权重为（0.6，0.3，0.1），如果 JIT 违规率超过 100，修改权重为（0.5，0.2，0.3），否则是（0.7，0.2，0.1）。

5 算法设计

我们在晶圆模块选择上加入了混合启发式蚁群算法来选择更优路径，并通过动态调参的方法来提高算法的自适应性。每片晶圆的流转控制作为独立任务提交至线程池，由线程池中的工作线程统一调度与执行。此仿真系统采用线程池并发方式来提升整体调度效率，并结合图算法和代价模型实现死锁的检测与预防。

5.1 多线程并发机制

5.1.1 锁的设置

系统中每片晶圆的加工任务由线程池统一调度和执行，线程池中的工作线程负责分配和运行多个晶圆的加工流程。

具体机制：

将每片晶圆的加工流程拆分为任务，提交至线程池中，由线程池的固定数量工作线程并行处理。

- 模块锁：LP、AL、LL、PM、TM 模块均由信号量控制互斥访问。
- 全局数据锁：MoveList 与 GanttData 使用独立的 Lock 进行保护，防止多线程同时写入导致数据错乱。

通过上述机制，系统在保持资源占用正确性与一致性的同时，减少了线程创建与销毁开销，实现了高效的并发仿真调度。

5.1.2 仿真流程概述：

出片阶段：

TM 从 LP 取片——>送入 AL 校准——>转移至 LL 模块准备进入真空环境

加工阶段：

双臂机械手从 LLA/LLB 中取片根据相应的路径通过蚁群算法的选择进入 PM 模块加工，每个 PM 采用 FIFO 排队、禁止超片，之后放到 LLD 冷却后回到 LLA/LLB。

回片阶段：

TM1 从 LLA/LLB 中取出回送到原 LP 中。

5.2 智能调度策略-混合启发式蚁群算法

为进一步提升系统调度效率并严格满足 JIT 约束，本文在路径调度中引入了混合启发式蚁群算法（ACO）策略。蚁群算法是一种模拟蚂蚁觅食行为的群体智能优化方法，具备并发搜索与正反馈等特性，能够在复杂解空间中快速收敛至高质量解 [9]。结合本系统的多线程调度架构，ACO 在多个候选加工路径上并行搜索，通过信息素引导与启发式规则计算出最优 PM 加工路径，以最小化总调度时间、避免死锁并降低 JIT 违规率。此外，算法在搜索过程中对已遍历的路径进行记录与标记，确保后续迭代中不重复选择相同路径，从而减少无效计算并防止陷入局部最优。

5.2.1 状态转移概率（调度选择规则）

蚂蚁在选择加工模块 PM 处理晶圆 w 时的概率:

$$P(\mathrm{PM}_k|\mathrm{W}_{ij}) = \frac{[\tau(PM_k)]^{\alpha} \cdot [\eta(PM_k)]^{\beta}}{\sum_{n \in Candidates} [\tau(PM_n)]^{\alpha} \cdot [\eta(PM_n)]^{\beta}}$$

其中：

- $\tau(PM_k)$： PM_k 的当前信息素强度

  PM_k 的启发函数：

$$\eta(PM_k) = \frac{1+\mathrm{Urgency(w)}}{1+(t_{completion} - t_{now})} \cdot bonus_i$$

若 PM 是空闲状态，bonus=1，否则 bonus=0.1。该启发函数用于衡量每个加工模块的调度优先级，通过综合其 jit 紧急度、预计完成时间判断是否可以立即使用和当前空闲状态，优先选择能尽快完成加工且当前处于空闲状态的 PM，以提高资源利用效率并降低晶圆等待时间和 JIT 违规风险。

- α：信息素影响因子
- β：启发函数影响因子

当信息素浓度越高、预计完成时间越早且当前处于空闲状态的 PM，被选中的概率越高。

5.2.2 信息素更新公式

蒸发阶段：

$$\tau(PM_k) \leftarrow (1-\rho) \cdot \tau(PM_k)$$

其中 $\rho \in [0.05, 0.3]$ 为信息素挥发系数。其作用是模拟信息素自然挥发，可以避免蚂蚁信息素的无限积累，进而使蚂蚁能根据外部环境的变化而变化 [9]。

增强阶段（强化路径）：

$$\tau(PM_k) \leftarrow \tau(PM_k) + \Delta\tau(PM_k)$$

其中：

$$\Delta\tau(PM_k) = \frac{Q}{C}$$

代表 PM 获得的信息素奖励。

- Q:常数，用于调节信息素增加量
- C：综合多目标（总生产时间越短、吞吐量越高、JIT 违规越少）后的得分

  其作用是当路径的综合评分 C 越小（表示性能越好），PM 获得的信息素奖

励越大，使得优质路径的信息素浓度更高，最终形成正反馈闭环。

5.2.3 动态调参机制

（1）性能指标：

为实现对调度系统运行状态的全面监控与自适应调整，本文引入了三项关键性能指标，用于驱动 ACO 参数的动态调节。

系统负载率：

为刻画系统在任意时刻的运行压力，引入了**系统负载率** $L \in [0,1]$：

$$L=0.5\times(\frac{busy_pm}{total_pm})+0.3\times(\frac{busy_ll}{total_ll})+0.2\times(\frac{busy_tm}{3})$$

该指标的计算采用加权综合评估方法，通过整合 PM 模块、装载锁 LL 和传输模块 TM 三类关键资源的实时状态来反映整体系统压力。该公式赋予不同资源差异化的权重系数，其中工艺模块负载占比 50%，装载锁负载占比 30%，传输模块负载占比 20%，这种权重分配体现了 PM 作为核心加工单元的主导地位。这种多维度加权计算方式既能准确捕捉瓶颈资源，又可避免单一指标评估的片面性。

JIT 违规率：

$$P_{JIT}=\min(1.0,\frac{Total_JIT_violations}{N_{\text{check}_JIT}+10^{-6}})$$

该指标表示当前调度过程中违反 JIT 时间约束的比例。其中，为了避免出现除零异常情况，分母中加入了一个微小的正数。该比值被归一化至[0,1]区间，越接近 0 表示调度策略在时序控制方面越理想。

死锁频率：

$$F_{deadlock}=\frac{N_{deadlock}}{\max(1,elapsed_time)}$$

该指标表示单位时间内系统发生死锁的平均次数。分子为系统运行期间累计触发的死锁次数，反映资源冲突的严重程度，分母为从仿真启动到当前时刻的总运行时间，通过 max(1, time) 避免除零。该值越高表明资源冲突越严重，需优化调度策略或锁机制。

（2）参数调整：

信息素重要性：

$$\alpha=\alpha_0+0.6L+0.4\cdot P_{JIT}-0.2F_{deadlock}，\alpha_0=1.0，\alpha\in[0.8,2.0]$$

当系统高负载且 JIT 违规严重时，α→2.0，算法更倾向于选择历史优质路径；当死锁发生率高时，α→0.8，增强随机探索。

启发函数权重：

$$\beta = \beta_0 - 0.5L，\beta_0=2.0，\beta \in [1.5, 2.5]$$

系统处于低负载时，权重依赖设备状态等实时信息进行探索；处于高负载时，权重平衡历史信息与实时状态。

挥发率：

$$\rho = \rho_0 + 0.1 \cdot L + 0.05 \cdot F_{deadlock}，\rho_0=0.1，\rho \in [0.05, 0.3]$$

当系统处于高负载时则加快信息素挥发来避免路径固化；当死锁发生时则提高挥发率以快速遗忘问题路径。最低挥发率为 5%（保留基础学习能力），最高挥发率为 30%（极端情况下快速重置路径信息）。

（3）动态探索：

若系统在一段时间内未出现性能提升（超过 10s），则引入激励项:

$$\Delta t = t - t_{start}$$

$$若\Delta t > 10 \begin{cases} \delta_{boost} = \min(0.5, \dfrac{0.1 \cdot \Delta t}{10}) \\ \beta = \beta + \delta_{boost} \\ \rho = \rho \cdot (1 + \dfrac{\delta_{boost}}{2}) \end{cases}$$

通过此激励项实现前 10s 保持原参数收敛；10-60s 线性增强探索；超过 60 保持最大探索强度。

5.3 混合调度策略

根据 JIT 限制我们定义了晶圆 w 的紧急度：

$$Urgency(w) = \min(1.0, \frac{t_{stay}^{m}(w)}{JIT_threshold \times k})$$

- k：缓冲系数（设为 0.8，即提前预警）

当 $Urgency(w) > 0.7$ 时触发预警机制，即快要到达 JIT 限制时间。

根据模块当前的状态定义晶圆优先级：

$$\Pr iority(w)=\begin{cases} P_{base}\cdot 0.8,\text{晶圆正在}PM\text{加工} \\ P_{base}\cdot(1+Urgency)\cdot 1.2\text{，PM加工完成等待取出} \\ P_{base}\cdot(1+Urgency)\text{，等待进入}PM \\ P_{base}\cdot 0.7\text{，在冷却中} \\ P_{base}\cdot 1.5\text{，冷却完成待取出} \\ P_{base}\cdot(1+Urgency)\text{，其他情况} \end{cases}$$

Pbase 指当前模块的基础优先级，机械臂从已经加工好的晶圆中根据晶圆的紧急度和优先级选择更合适的晶圆进行转移。

5.4 死锁检测与预防

在多模块高耦合的晶圆调度系统中，由于缓冲区受限、资源共享（如阀门互斥）及 JIT 时间限制等多因素的交互，系统极易陷入死锁状态。为此，本文提出一种多阶段死锁解决机制，包括：死锁环检测、牺牲晶圆选择、晶圆回滚恢复。该机制可在不中断主流程的前提下自动清除死锁，提升系统鲁棒性。

5.4.1 构建资源分配图

首先需要为晶圆与模块构建资源分配图 $G=(V,E)$，其中：

V：表示系统中所有正在等待资源的晶圆。

边 $(w_i \to w_j)\in E$：表示晶圆 w_i 正在等待一个由 w_j 占用的资源。

检测死锁的整体过程如下：

1）资源占用状态收集：对于每个独占资源（如 PM 模块、LL 槽位等），记录当前被哪个晶圆占用。

2）构建依赖关系：系统通过晶圆路径动态分析每个晶圆的下一目标模块。若该模块正被其他晶圆占用，则加入依赖边。

3）环路检测：基于深度优先搜索（DFS）方法检测图中是否存在环路 $C\in G$。若存在，则说明系统当前已陷入死锁，提取该环中所有晶圆用于下一步牺牲判断。

5.4.2 基于牺牲函数的晶圆选择

为打破死锁，我们需从死锁环 C 中选择一个牺牲晶圆 w_v 执行回滚。选择依据如下牺牲代价函数：

$$S(w)=\alpha\cdot time(w)+\beta\cdot step(w)+\gamma\cdot(1\text{-Priority}(w))+\delta\cdot(1-Urgency(w))$$

$time(w)$ 表示晶圆已加工的时间，$step(w)$ 表示晶圆剩余加工步数，Priority(w) 表示晶圆占用资源的优先级，$Urgency(w)$ 表示 JIT 紧急程度。我们定义权重 $(\alpha,\beta,\gamma,\delta)$ 为 $(0.5,0.4,0.3,0.2)$ 。

最终选择代价最小的 w^* 作为牺牲目标进行回滚：

$$w^* = \underset{w \in DeadlockCycle}{\arg\min} \ (S(w))$$

5.4.3 晶圆回滚机制

回滚操作包括以下两步：

1）强制中止加工：若目标晶圆当前处于 PM 模块中，立即终止加工并释放资源。

2）返回最近安全 LL 槽位：从路径列表中反向查找该晶圆最近经过的 LL 模块，若目标模块为真空区域的 LLC 或 LLD，则优先尝试将晶圆放入其槽位；若为 LLA 或 LLB，则优先尝试放入 S2 槽位，以尽量避免与正常调度路径冲突。若目标槽位可用，系统直接完成回退操作；若 S2 槽位被占用，系统尝试将占位晶圆移回其原始 LP 模块；若无法完成回退，则在当前死锁环中重新评估牺牲代价，选取代价最小的晶圆作为新的回退对象，递归执行上述流程。

5.5 算法整体实现流程概况图

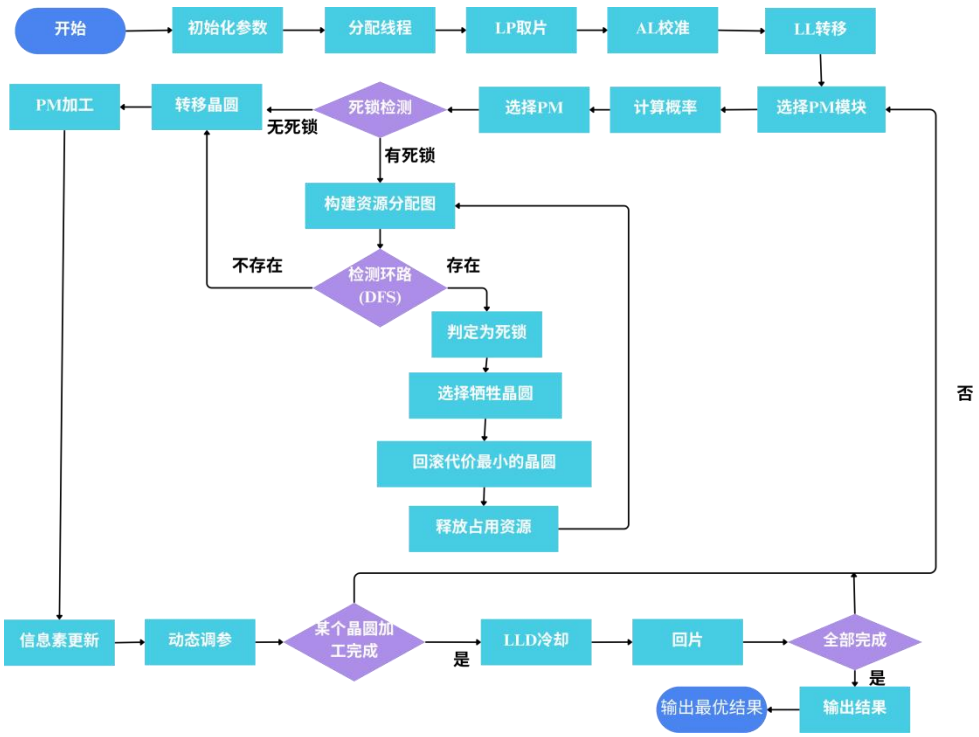

图 2 算法流程图

6 求解结果

由于蚁群算法的随机性特征，本文的结果为 10 次独立实验的平均值，其标准差控制在±5%以内。为了加快晶圆仿真的速度，代码运行时我们将时间缩小了 1000 倍，但本文给出的结果是实际求解的晶圆调度时间，json 数据在技术数据总压缩包中。

6.1 任务 1

6.1.1 任务 a

总处理时间 52280s（jit 违规次数 78）

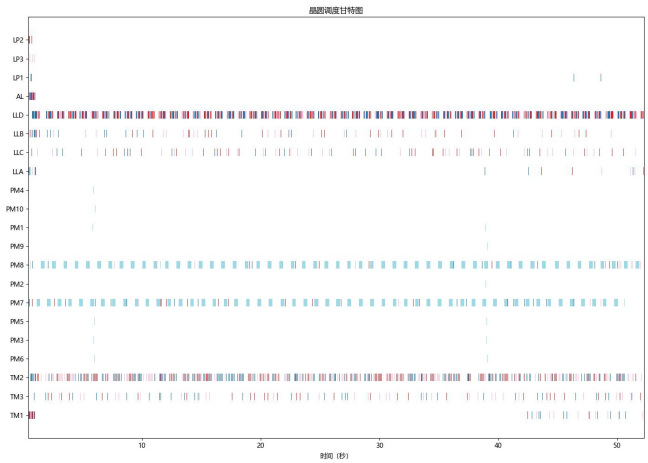

图 3 1-a 的结果 Gantt 图

6.1.2 任务 b

总处理时间 28540s（jit 违规次数 5）

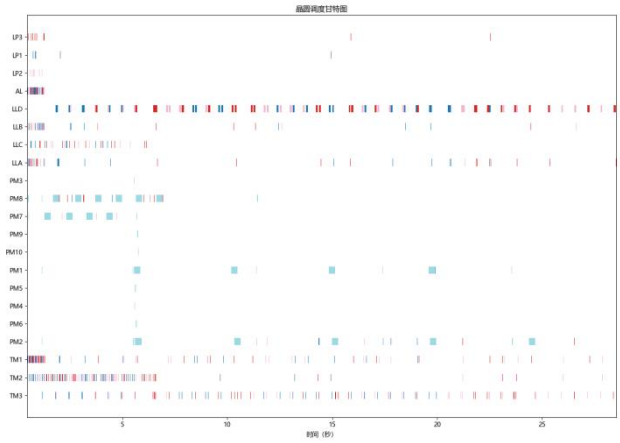

图 4 1-b 的结果 Gantt 图

6.1.3 任务 c

总处理时间 14590s（jit 违规次数 57）

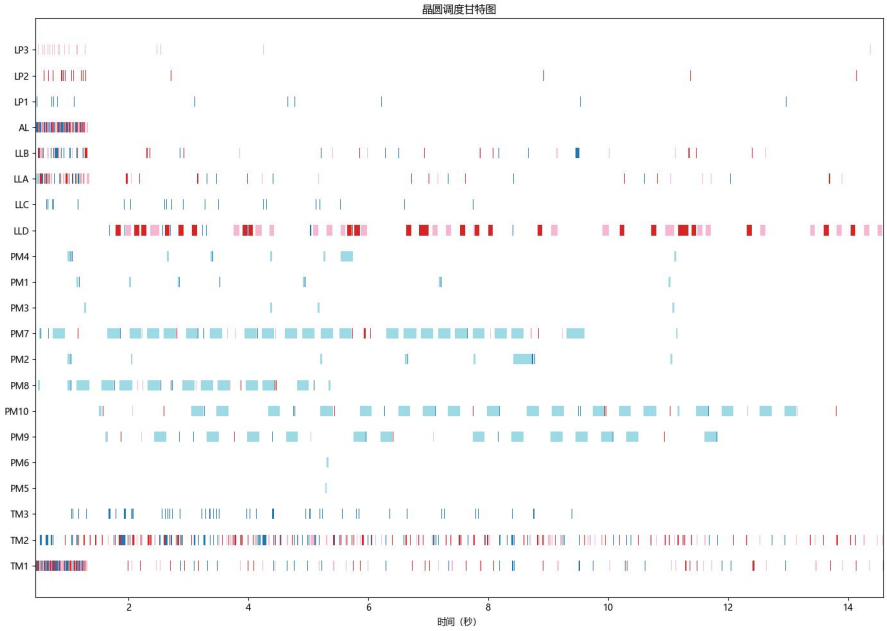

图 5 1-c 的结果 Gantt 图

6.1.4 任务 d

总处理时间 15670s（jit 违规次数 31）

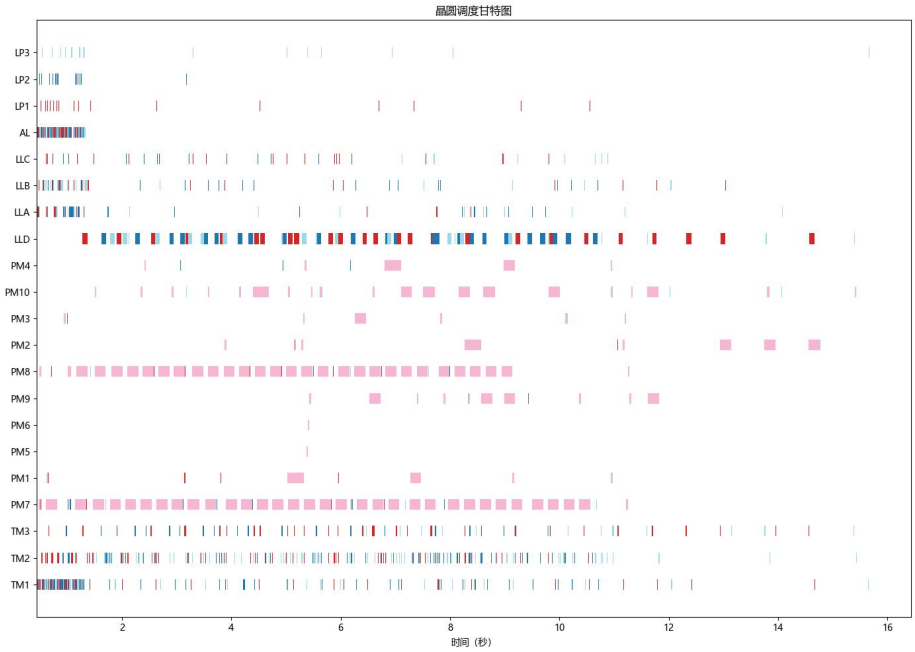

图 6 1-d 的结果 Gantt 图

6.2 任务 2

6.2.1 任务 a

总处理时间 19710s（jit 违规次数 124）

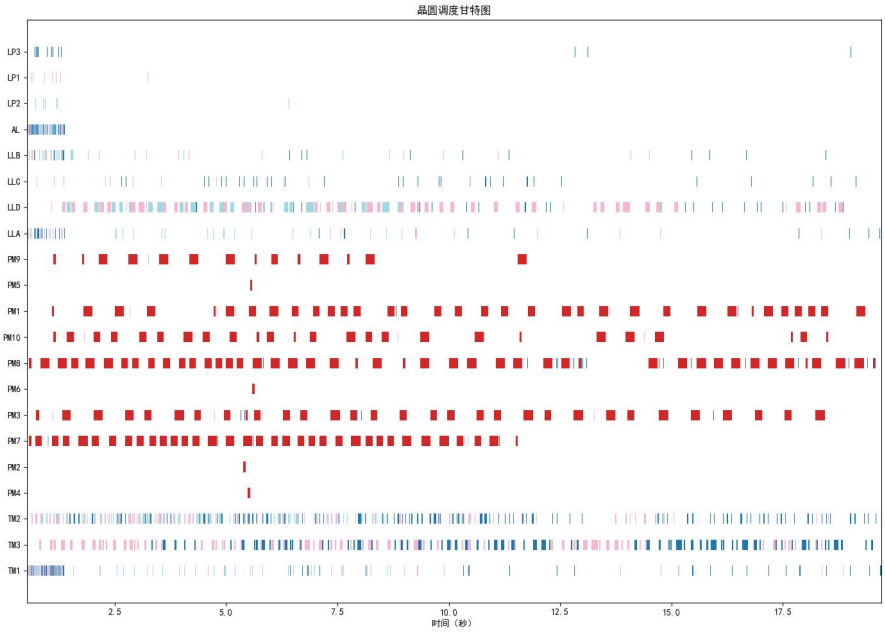

图 7 2-a 的结果 Gantt 图

6.2.2 任务 b

总处理时间 38760s（jit 违规次数 123）

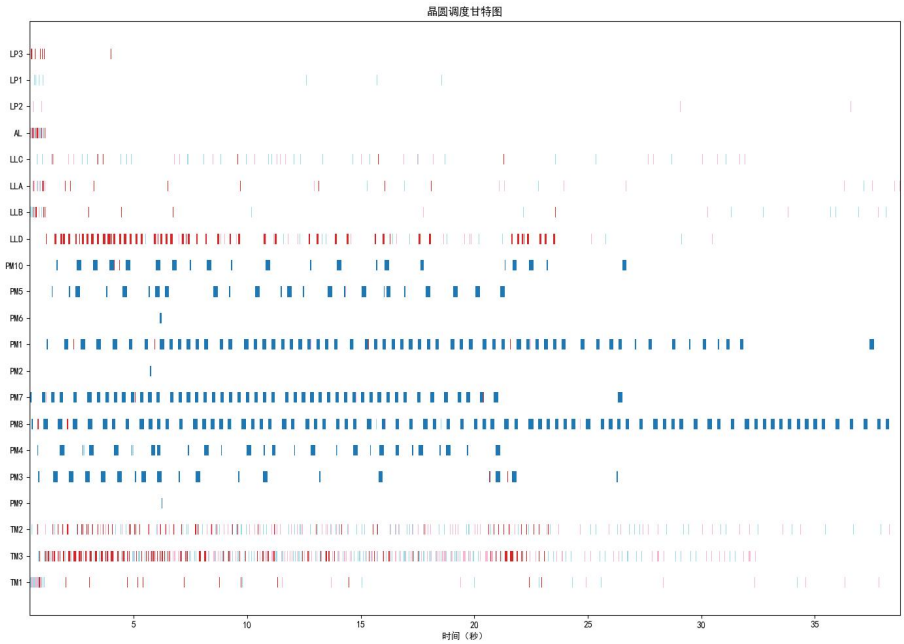

图 8 2-b 的结果 Gantt 图

6.2.3 任务 c

总处理时间 19120s（jit 违规次数 64）

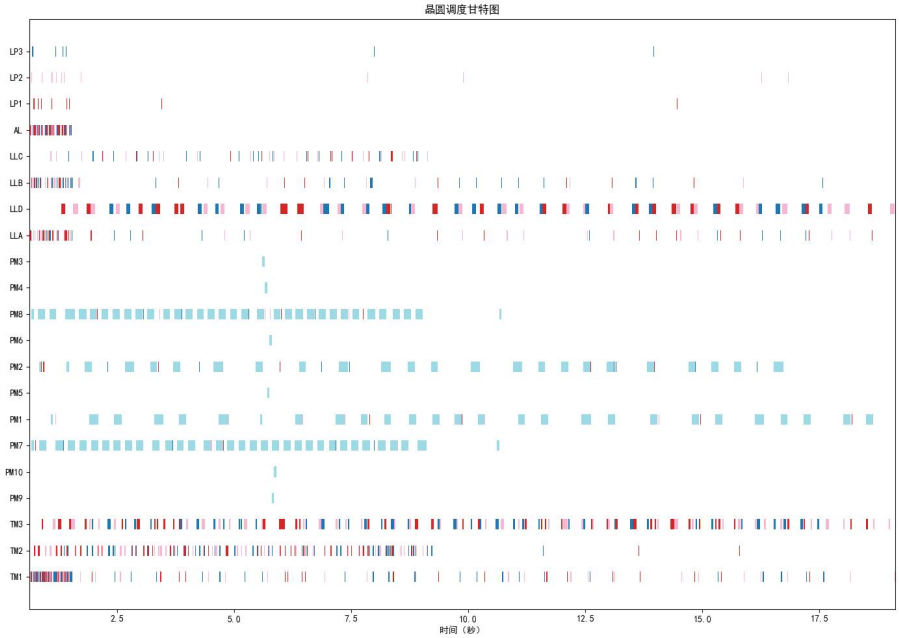

图 9 2-c 的结果 Gantt 图

6.2.4 任务 d

总处理时间 23350s（jit 违规次数 76）

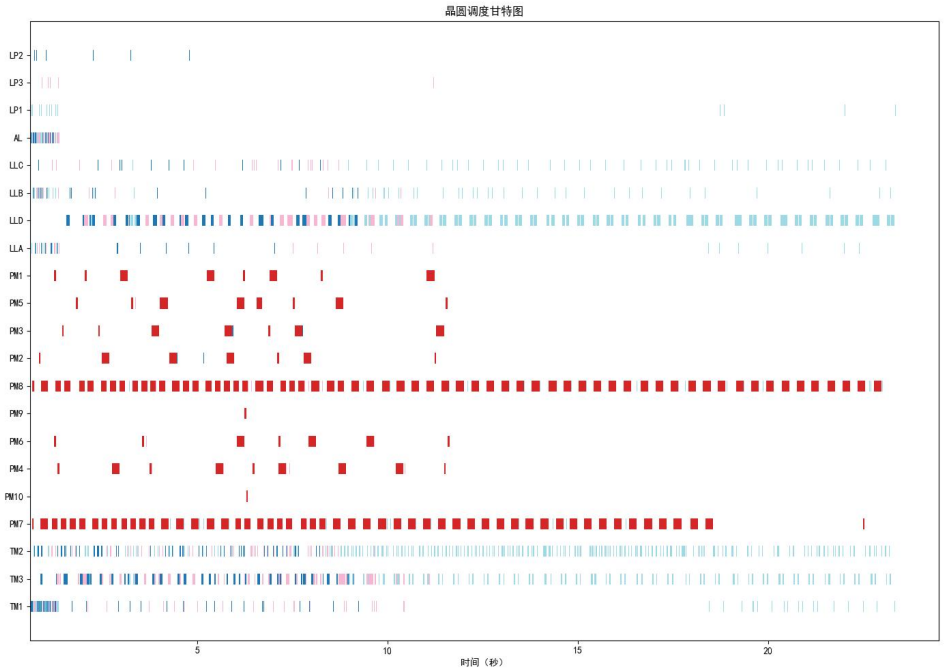

图 10 2-d 的结果 Gantt 图

6.3 参数自适应结果分析(以任务 2 的 d 为例)

6.3.1 实时调度监控图曲线说明

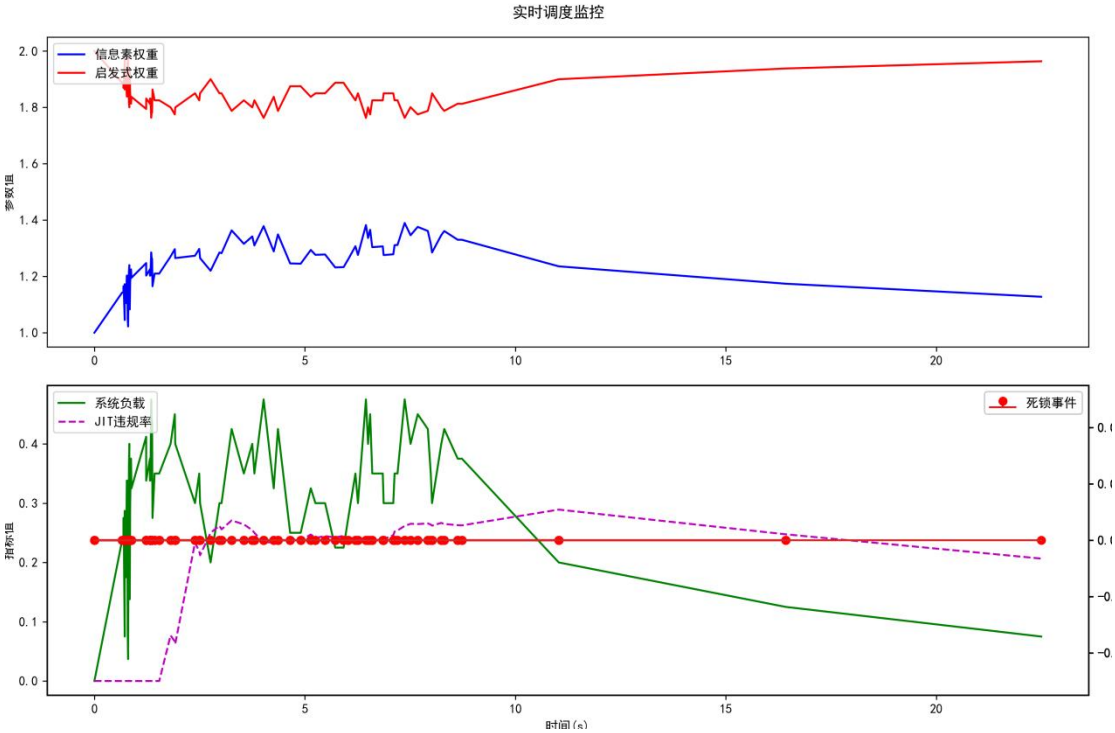

图 11 2-d 的实时调度监控图

系统状态监控

系统负载（绿色）：

曲线在初期迅速升高，并且在接下来的运行中的波动剧烈，则表明大部分模块处于忙碌状态，也就是说调度系统的资源利用率高，后期加工的晶圆减少，负载趋于稳定。

JIT 违规率（紫色虚线）：

初期迅速上升，并不断波动，说明系统还未调整好晶圆调度参数，随后缓慢下降趋于稳定，表现了经过算法的不断优化，逐渐满足 JIT 违规限制。

死锁事件（红色点+红线基准）：

图中红点表示每次死锁检测到的事件数，红线为基准参考值。该曲线表明死锁在仿真过程中零星出现，但始终在极低频率范围内，并得到及时处理，未造成系统阻塞。

6.3.2 ACO 参数自适应过程图曲线说明

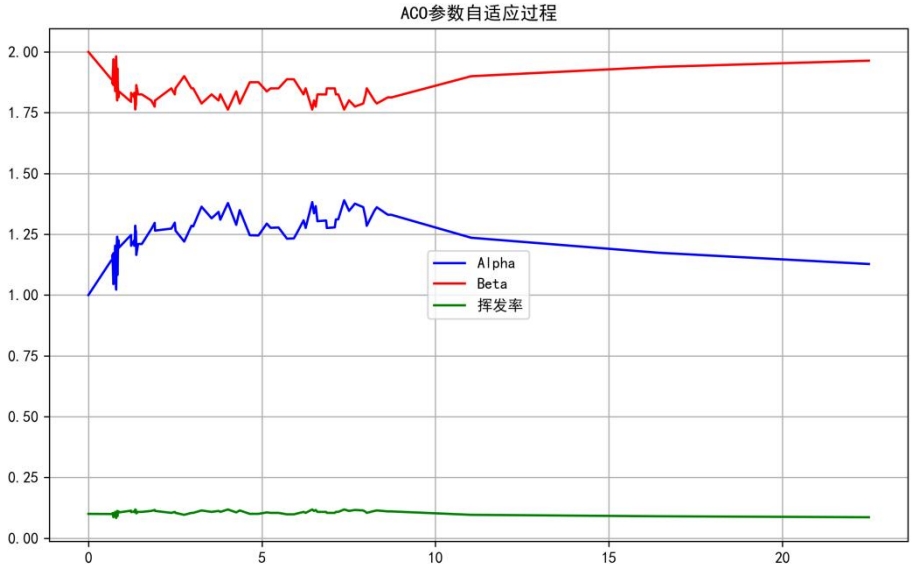

图 12 2-d 的 ACO 参数自适应过程图

Alpha（蓝线）：

表示信息素的影响因子。从初始值 1.0 迅速波动上升，后期略微下降，表明在最初阶段此因素会使调度快速建立路径依赖，到了后期则更偏向于在负载均衡当中回归来保证调度的搜索能力。

Beta（红线）：

表示启发式函数对路径选择的影响强度。初始值为 2.0，曲线在不断的波动，随后稳定在 1.8 左右，这意味着随着系统运行过程中不断测试各种不同的路径后，将最后趋于利用启发式信息做决策。

挥发率（绿色）：

表示信息素挥发速度。曲线整体变化平缓，且处于小幅波动状态，说明该系统带有历史的经验并且能在一定程度上进行信息素的遗忘，用于保持路径学习的速度稳定。

由实验结果可知，调度系统可在投入运行后快速完成各参数的初始化和根据当前负载情况进行动态调整，并始终保持较好的性能，且 ACO 的核心参数可随系统负载、死锁率及路径分布的情况进行动态调整并趋于稳定，实现了晶圆调度控制的合格、稳定、高效。另外，系统负载曲线与 Alpha、Beta 动态匹配，说明本系统在智能调度与实时调参方面具备良好的鲁棒性和自适应性。

6.4 多目标优化过程中目标函数的演化趋势

为验证此算法在半导体调度中的多目标优化能力，本文设计了以总时间（f1）、吞吐量（f2）与 JIT 违规次数（f3）为目标函数的实验。从八个任务结果中我们挑选了在不同迭代轮数下，各目标函数的演化趋势，图 13 是任务 1 的任务 d 目标函数的演化趋势图，迭代次数，图 14 是任务 2 的任务 d 目标函数的演化趋势图。

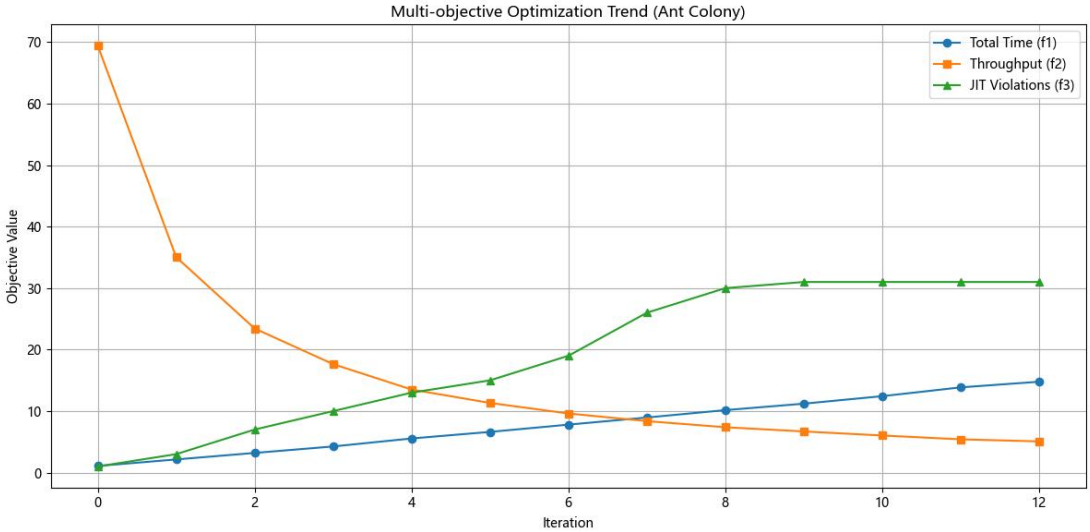

图 13 1-d 目标函数的演化趋势图

图 13 的总时间 f1 呈直线上升，说明在调度过程中，为保证系统能提高吞吐量、降低 JIT 违规，则需要花费更多的时间；目标 f2 快速下降，说明蚁群算法逐步找到更有效的搬运路径，显著提升了单位时间内处理晶圆的效率；JIT 在整个过程中逐步上升且越来越缓慢，最后稳定，表示系统在逐步优化晶圆的调度过程，尽量满足 JIT 的限制。

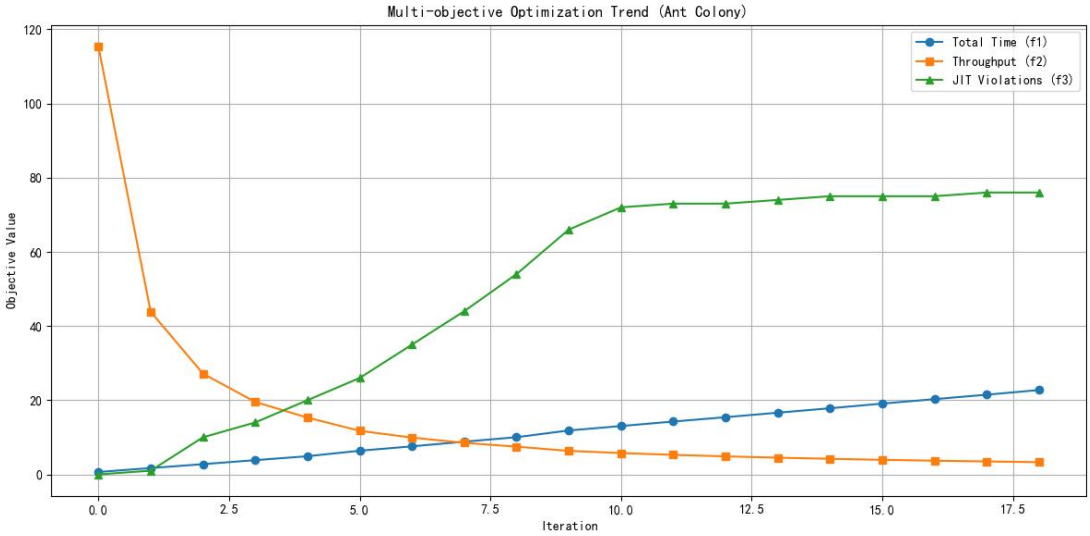

图 14 2-d 的目标函数的演化趋势图

图 14 的总时间持续上升，说明算法在后期更多倾向于保证其他目标；吞吐量下降趋势逐渐缓慢，可能是探索范围更广；JIT 在整个过程中逐步上升且越来越缓慢，随后稳定，表示系统在逐步优化晶圆的调度过程来满足 JIT 的限制。

7 算法性能分析

本文所设计的半导体晶圆调度仿真系统综合考虑了模块的资源互斥、双机械臂调度、Just-in-Time 时间约束、状态切换逻辑、路径智能优化（ACO）以及死锁检测与回滚机制。

假设：晶圆总数为 W、PM 模块数量为 P、蚂蚁数量为 A、信息素搜索迭代次数为 TS、每个晶圆路径长度约为 S、总晶圆移动次数为 V、死锁检测周期为 Td。

7.1 主控调度流程

晶圆按照预设工艺路径顺序流转加工。单片晶圆的计算复杂度包括路径查询 O(S)、各步骤调度判断与状态锁控制约为 O(1)，在 W 片晶圆的情况下，总复杂度为：

$$O(W \cdot S)$$

7.2 机械臂调度

单机械臂和双机械臂都是每次仅能处理一个动作，晶圆在路径转移时机械臂都要进行取片、搬运和放片，因此整体机械臂搬运调度时间复杂度为：

$$O(W \cdot S)$$

7.3 ACO 蚁群算法调度

对于每个晶圆存在多个候选 PM 模块路径节点，系统使用改进的混合启发式蚁群算法进行调度选择。假设蚂蚁数量为 A，每条路径中候选模块数为 P，信息素搜索迭代次数为 TS，则路径选择的时间复杂度为：

$$O(W \cdot A \cdot P \cdot TS)$$

7.4 死锁检测机制

遍历所有晶圆 O(W)，查询路径确定下一个资源模块 O(S)，构造等待图和 DFS 检查环 $O(W^2)$；遍历死锁晶圆数量为 D（远小于 W），每个死锁晶圆都要计算处理的时间，因此计算牺牲代价 O(DS)；执行回滚反向遍历路径 O(V)，因此整体复杂度：

$$O(W^2 + D \cdot S + V)$$

7.5 Just-in-time 检查

系统在每次晶圆驻留与传输阶段执行 JIT 时间检查，因此整体为：

$$O(W \cdot S)$$

7.6 整体复杂度

系统最坏的时间复杂度为上面几个部分相加，但实际中，A、P、S、V、D、TS 为常数，其中主导项取决于晶圆数量 W 与路径平均长度 S。因此整体复杂度可简化：

$$O(W^2 + W \cdot S)$$

8 算法优化前后对比

优化前，调度路径依赖固定规则，未使用智能化优化算法，其特点调度策略缺乏灵活性，无法根据系统状态动态调整，模块之间存在资源竞争与等待现象，整体生产时间较长，对 JIT 约束缺乏有效控制，违规次数偏多。因此，该阶段的调度结果整体耗时较高。例如，任务 1-a 的总生产时间为 78080 秒，任务 2-b 的总生产时间为 42390 秒。

优化后，引入了混合启发式蚁群算法（ACO），并在此基础上进行了多方面优化：

- 路径选择优化：利用信息素浓度与启发函数相结合，根据模块的空闲状态、预计完成时间以及晶圆的紧急度动态选择加工路径。
- 动态调参机制：引入系统负载率、JIT 违规率与死锁频率等指标，对 ACO 参数进行实时调整，以实现自适应优化。
- 多目标优化：综合考虑最小化总生产时间、最大化吞吐量与最小化 JIT 违规次数，并根据 JIT 情况动态调整目标权重。
- 死锁预防：在牺牲代价模型基础上，引入资源依赖图和晶圆回滚机制，可有效检测并解除死锁，保证系统运行的稳定性。

经过优化后，调度性能显著提升，例如：

任务 1-a 总时间从 78080 秒 → 52280 秒，缩短约 33%；

任务 1-c 总时间从 19740 秒 → 14590 秒，缩短约 26%；

任务 2-a 总时间从 24300 秒 → 19710 秒，缩短约 19%。

任务 2-d 总时间从 26700 秒 → 23350 秒，缩短约 13%。

通过引入混合启发式蚁群算法，系统在整体生产时间、JIT 合规性、死锁处理能力与稳定性方面均获得了显著提升。相较固定规则调度，优化后的方案不仅显著缩短了整体调度时间（平均缩短约 20%），还提升了算法的鲁棒性与自适应性，为半导体智能调度提供了更优的解决方案。

9 模型改进

1、现在使用的 ACO 算法为固定迭代次数方式，在面对高复杂度问题时耗时较长。

2、现在还存在 JIT 违规的情况，晶圆长时间滞留可能会引发死锁。

3、该文章仅展现了本算法的结果，未与其他方法进行对比，无法明确的判断该算法是否是更优的算法。

10 引用文献

[1]郭乘涛.基于问题分解与蚁群算法的半导体晶圆制造系统调度方法的研究[D].上海交通大学,2012.

[2]胡晟熙,宋日荣,陈星,等.云边协同计算中基于强化学习的依赖型任务调度方法[J].计算机科学,2023,50(S2):712-719.

[3]姜佳琪.考虑速率维修和资源分配的单机时间窗调度问题[D].浙江工商大学,2023.DOI:10.27462/d.cnki.ghzhc.2023.000450.

[4]杨雯惠,陈璐,张昕莹.考虑设备时变效应的晶圆加工序列决策调度方法[J].哈尔滨工业大学学报,2022,54(07):29-36.

[5]赵坤.多品种晶圆混合加工的双臂组合设备调度分析[D].江西理工大学,2018.

[6]黄国新,林文文,章天吉.基于遗传算法的含真空锁状态约束的集束型设备智能调度[J].装备制造技术,2022,(10):76-79+87.

[7]付虎,伍乃骐,乔岩.半导体制造中具有重入加工工艺的双臂组合设备的调度研究[J].机电工程技术,2013,42(09):32-37.

[8]王君舰.基于蚁群算法的 GARCH 模型的人民币—美元汇率预测研究[D].哈尔滨工业大学,2009.

[9]夏亚梅,程渤,陈俊亮,等.基于改进蚁群算法的服务组合优化[J].计算机学报,2012,35(02):2270-2281.